



Universidad Abierta Interamericana

Facultad de Tecnología Informática

Ingeniería en Sistemas Informáticos

TRABAJO FINAL DE INGENIERÍA

Manual de programador

Arquitectura, patrones y reglas para modificar el sistema

Alumno: Pettini, Patricio Ezequiel

Legajo: B00072710-T1

Sede: Sede Norte — Boulogne

Docentes: Romero, Martín Miguel · Weingand, Gastón Matías

Versión 1.0 · 17/09/2026

Índice

Alcance de esta versión	4
Alcance	5
Stack	5
Estructura de la solución	5
Reglas de capa	6
Patrones aplicados	7
Las dos bases de datos	7
Aislamiento entre organizaciones	9
Métricas	9
Seguridad	10
Motor predictivo	11
Dos decisiones que hay que respetar al modificarlo	11
Cómo se calcula el score	11
Las cinco reglas	12
Alertas	13
Integración de IA	13
Ninguna implementación lanza ante un fallo del servicio	13
El origen de la decisión se registra con tres valores	13
Disyuntor	14
Flujo de registrar una incidencia	14
Frontend	15
Pruebas	16
Al modificar el sistema	16
El ciclo de vida de la incidencia	17
El reparto entre validar y guiar	18
La fecha de resolución	18
Reportes en PDF	18
Al agregar una patente	19

Al agregar un código de evento a la bitácora	19
Fuentes.....	21

Alcance de esta versión

Este manual describe la versión **1.0** del sistema, del **17/09/2026**, que es la que se entrega con el Trabajo Final. Documenta lo que el sistema hace hoy, no lo que va a hacer: si una pantalla no aparece acá, es porque todavía no existe.

Al cierre de esta versión el backlog registra **90 de 92 historias terminadas**. Lo que queda abierto está declarado, con su sprint asignado, en el documento de gestión del proyecto.

Este manual es para quien va a mantener o extender el código. No cubre el uso del sistema: eso está en los tres manuales de usuario.

Sistema: PredictIT — Gestión y mantenimiento predictivo de equipos informáticos

Versión: 1.0

Destinatario: quien tenga que mantener o extender el sistema.

Alcance

Este manual describe cómo está construido el sistema: sus capas, sus patrones, sus decisiones de diseño y los lugares donde hay que tener cuidado al modificarlo. Las decisiones con alternativas evaluadas están registradas aparte, en docs/adr/, en formato Nygard; acá se describe el resultado.

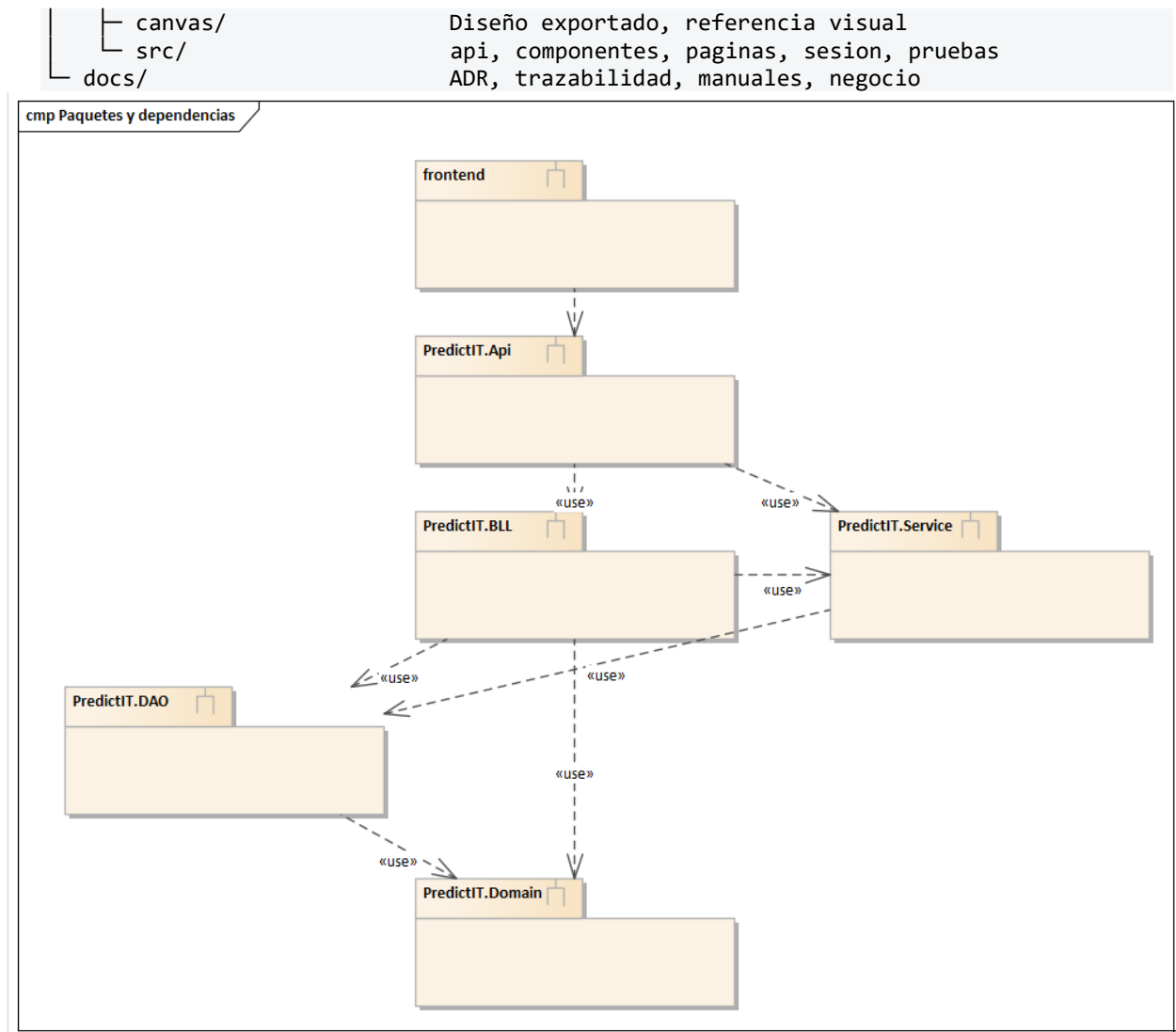
Stack

Capa	Tecnología
Frontend	React 19, TypeScript, Vite
API	ASP.NET Core 9, C# 13
Acceso a datos	ADO.NET con DAO explícito
Base de datos	SQL Server 2022 en contenedor
Pruebas	xUnit y FluentAssertions; Vitest en el frontend

El documento original especificaba Java, Spring Boot y MySQL. El cambio a C# está justificado en el ADR 0002: es el lenguaje de la carrera y el del proyecto de referencia cuya separación de capas evalúa la cátedra.

Estructura de la solución

TFI/	
├─ arrancar.ps1	Levanta todo
├─ docker-compose.yml	SQL Server 2022
├─ db/	Esquemas y datos iniciales
├─ backend/	
│ ├─ PredictIT.Domain/	Entidades. No referencia a nadie
│ ├─ PredictIT.DAO/	Único que conoce SQL
│ ├─ PredictIT.BLL/	Reglas de negocio, devuelve DTOs
│ ├─ PredictIT.Service/	Transversal: seguridad, bitácora, IA, motor
│ ├─ PredictIT.Api/	Controladores finos, JWT, filtros
│ └─ PredictIT.Tests/	362 pruebas
└─ frontend/	



Reglas de capa

Son lo que evalúa la cátedra y no son negociables:

- **`Api` sólo habla con `BLL`.** Nunca toca el DAO ni escribe SQL.
- **`BLL` orquesta, valida y decide.** Devuelve DTOs, nunca entidades crudas.
- **`DAO` es el único que conoce SQL.**
- **`Domain` no referencia a nadie.**
- **`Service` es transversal.** Lo consumen BLL y Api, nunca al revés.

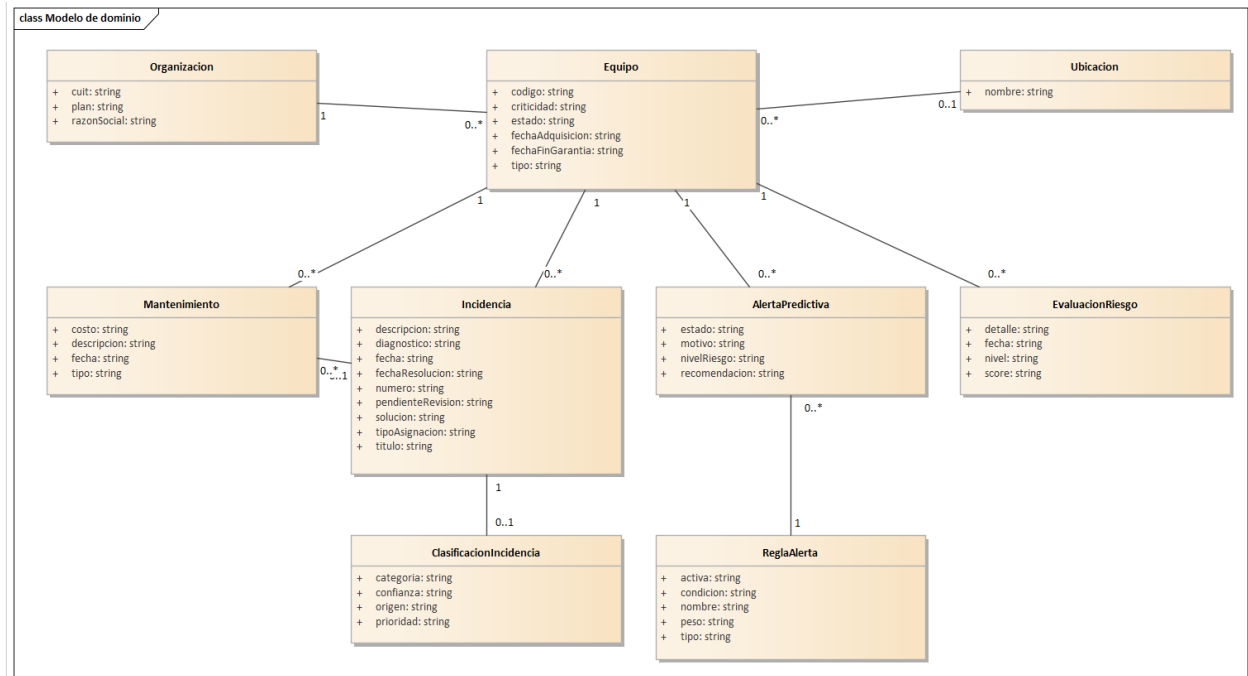
La validación vive en la BLL y no en el controlador, para que valga en cualquier punto de entrada. Los controladores además repiten el control de permisos: es una segunda barrera que corta antes de tocar la base y deja el rechazo en bitácora.

Patrones aplicados

Patrón	Dónde	Para qué
Abstract Factory	FactoryDao, FactoryDaoSeguridad, FactoryBusiness	La capa de arriba pide contratos y nunca nombra una clase concreta
Repository genérico	IGenericDao<T>	Operaciones comunes de persistencia
Data Mapper	IObjectMapper<T>	Traducir filas a entidades
Unit of Work	SqlTransactRepository	Varias operaciones en una transacción
Composite	Familia / Patente	Permisos anidados a profundidad arbitraria
Strategy	IEstrategiaRegla	Una estrategia por tipo de regla predictiva
Adapter	IProveedorIA	Aislar el servicio externo
Decorator	ProveedorIAConDisyuntor	Resiliencia sin tocar la implementación
Circuit Breaker	Disyuntor	Dejar de llamar a un servicio caído

Las dos bases de datos

PredictIT_Negocio tiene los datos operativos: organizaciones, equipos, incidencias, mantenimientos, planes y agenda del preventivo, comprobantes del servicio, reglas y alertas. PredictIT_Servicio tiene usuarios, roles, familias, patentes, bitácora, idiomas y respaldos.



Cuatro tablas de negocio que conviene ubicar porque son las últimas que entraron:

Tabla	Qué guarda	Detalle que no es obvio
PlanMantenimiento	Cada cuántos días le toca el preventivo	El alcance es por equipo o por tipo, nunca los dos: lo garantiza un CHECK que los cuenta con CASE, porque T-SQL no compara expresiones booleanas entre sí
MantenimientoProgramado	Una fecha concreta agendada	«Vencido» no es un estado: es PROGRAMADO con la fecha pasada. Guardarlo obligaría a un proceso diario, y el día que no corra el sistema mostraría datos viejos como ciertos
Comprobante	Lo que se le factura a la organización por período	Los números del plan quedan copiados : si se leyera del plan al consultar, cambiar el precio reescribiría los comprobantes del año pasado
LineaComprobante	El desglose del comprobante	El importe se guarda, no se recalcula al leer, por el mismo motivo

El correlativo del comprobante es único por organización, y va como **índice único filtrado** por numero IS NOT NULL y no como UNIQUE: SQL Server trata el NULL como un valor más en una restricción de unicidad, así que un UNIQUE sobre (id_organizacion, numero) deja tener un solo borrador por organización.

Están separadas de verdad, con **dos cadenas de conexión distintas**. Un DAO de negocio no puede leer la base de seguridad ni al revés: es el punto de la separación, no una formalidad.

Consecuencia práctica. Las referencias entre bases son GUID lógicos sin clave foránea. `Incidencia.id_tecnico` apunta a un usuario de la otra base y nadie garantiza que exista. Los datos que hay que cruzar se resuelven con dos consultas y un cruce en memoria, nunca con una consulta entre bases: eso las ataría a estar en el mismo servidor.

Y el N+1 acecha. Resolver el nombre del técnico de cada incidencia de a una dispararía una consulta por fila. En todos los listados el nombre se resuelve **por lote**: una consulta por el conjunto y composición en la BLL. Es un cuidado que hay que sostener en cada listado nuevo.

Aislamiento entre organizaciones

Es el riesgo de seguridad más serio del sistema. Una consulta que se olvide el filtro por organización filtra datos de un cliente a otro, y **ningún analizador estático lo detecta**.

Las tres reglas:

1. **El identificador de organización nunca viaja en la petición HTTP.** Se resuelve del token. Si viajara, un controlador podría pedir datos de otra organización.
2. **Todo DAO de negocio filtra por el contexto de sesión**, sin excepción y sin que el llamador pueda elegir otra organización. No existe un método que devuelva datos «de todas las organizaciones».
3. **Las altas fuerzan la organización del contexto**, ignorando la que traiga la entidad.

Hay una prueba de aislamiento **por operación**, no una de muestra, porque el defecto se introduce de a una consulta.

Cuidado adicional: las tablas que no tienen `id_organizacion` —`ClasificacionIncidencia`, `RecomendacionAsignacion`— dependen del *join* contra `Incidencia` para estar aisladas. Si ese *join* se cayera, alcanzaría el identificador para leer datos de otro cliente. Hay pruebas específicas para eso.

Métricas

`GET /api/metricas` devuelve el estado del sistema en formato Prometheus (ADR 0015). Tres grupos:

Métrica	Para qué
predictit_tiempo_en_linea_segundos	Es lo que hace verificable el RNF-05. Cada vez que el número baja, hubo un reinicio
predictit_disyuntores_abiertos	Circuitos hacia el proveedor de IA que no están cerrados, del ADR 0009
predictit_peticiones_limitadas_total	Lo que rechazó el limitador de velocidad

Más lo que aporta la biblioteca sin escribir nada: duración de cada petición por ruta y código —RNF-04— y memoria, GC e hilos del proceso —RNF-09—.

El endpoint no está por omisión. Se habilita configurando `Metricas:Token` —en el docker-compose es `PREDICTIT_METRICAS_TOKEN`— y exige ese token en la cabecera `X-Metricas-Token`. Sin token configurado devuelve 404, y con token equivocado también: un 401 confirmaría que el endpoint existe, que es justo lo que no conviene decirle a quien está probando.

Al agregar una métrica, va en `Api/Infraestructura/Metricas.cs`. Los gauges que hay que preguntar —como los disyuntores— se actualizan en `Actualizar()`, que corre al servir la página; los contadores se incrementan donde ocurre el hecho.

Seguridad

Contraseñas. PBKDF2-HMAC-SHA256, 600.000 iteraciones —la recomendación de OWASP—, formato `PBKDF2$iteraciones$salt$hash`. Las iteraciones viajan en el hash, así que subirlas no invalida las contraseñas existentes: se rederivan al entrar, con la contraseña que la persona acaba de escribir. La comparación usa `CryptographicOperations.FixedTimeEquals`. `Verificar` devuelve false ante una entrada malformada en lugar de lanzar.

Autenticación. JWT. El token lleva el usuario y la organización activa, pero **no las patentes:** se leen de la base en cada petición. Si vinieran en el token, revocar un permiso no tendría efecto hasta que expirara.

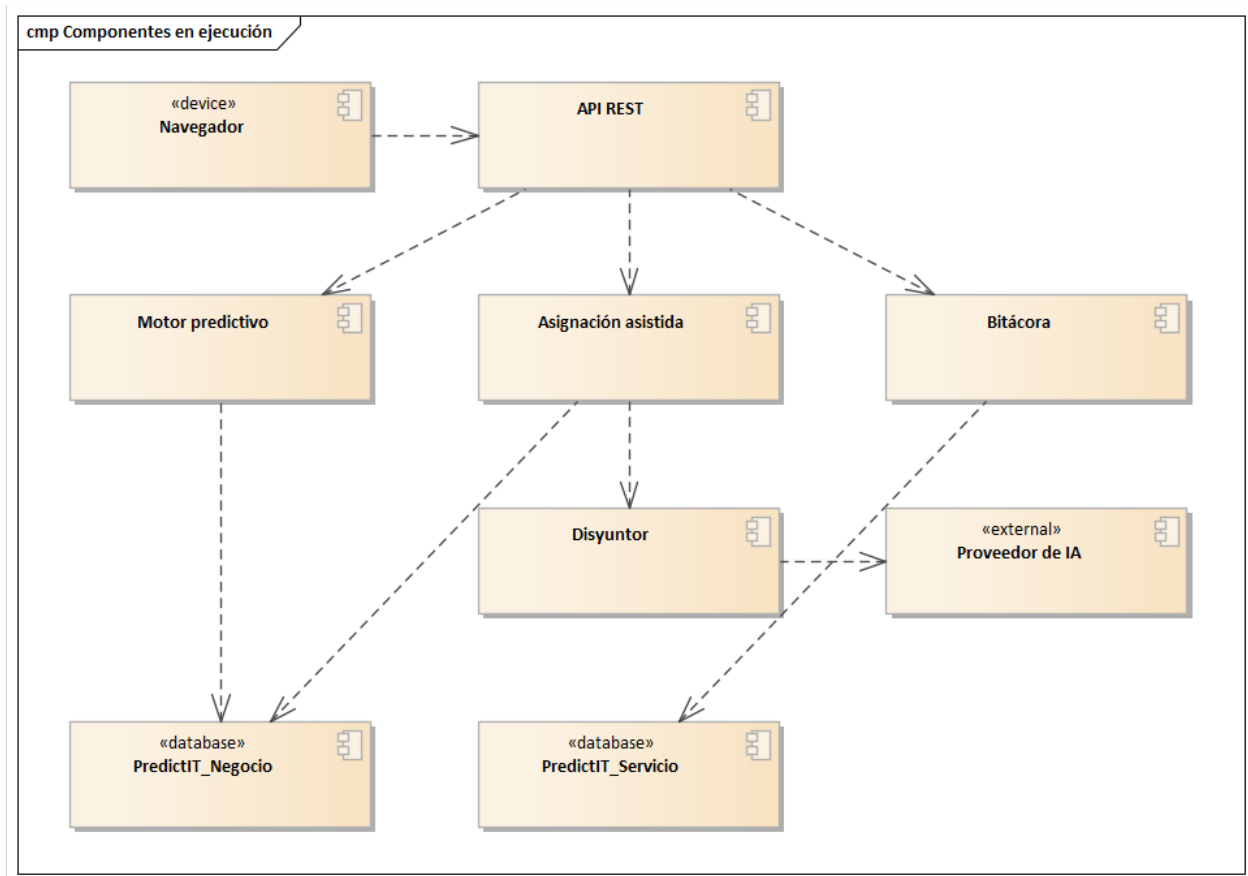
Autorización. Composite de familias y patentes. Familia puede contener patentes y otras familias, a profundidad arbitraria. La recolección de patentes lleva un conjunto de visitados que **corta ciclos:** sin él, una familia que se contiene a sí misma colgaría el proceso.

El atributo `[RequierePatente]` admite varias patentes y alcanza con tener una. Hace falta porque hay pares donde ninguna implica a la otra: `INCIDENCIA_VER_TODAS` y `INCIDENCIA_VER_PROPIAS` son distintas, y el administrador tiene sólo la primera.

Bitácora. Inmutable, garantizado en la base con un disparador `INSTEAD OF UPDATE`, `DELETE` que lanza un error. La restricción no depende de que el código la respete: la tabla no se puede modificar ni desde `sqlcmd`.

Motor predictivo

Está en PredictIT.Service/Prediccion/. Es **lógica pura, sin acceso a datos**: la BLL le arma el contexto y persiste el resultado.



Dos decisiones que hay que respetar al modificarlo

La fecha de referencia entra por parámetro. El motor no lee el reloj. Sin esto no existe caso de prueba posible: el resultado cambiaría según el día en que se corran las pruebas. Si se agrega una regla, tiene que tomar la fecha del contexto.

Una regla que no se puede evaluar se excluye del promedio ponderado, no aporta cero. Si aportara cero, un equipo con la ficha incompleta aparecería *menos* riesgoso que uno bien cargado, que es exactamente al revés.

Cómo se calcula el score

Promedio de las intensidades ponderado por el peso de cada regla, normalizado sobre las reglas aplicables. Se normaliza porque los pesos los configura la organización y nada garantiza que sumen 100.

Después se multiplica por un factor de criticidad del equipo, de 0,70 a 1,15. Es **multiplicativo y no aditivo** a propósito: así un equipo crítico sin ningún problema sigue dando cero.

Cortes: bajo hasta 39, medio de 40 a 69, alto de 70 a 100. Están definidos en un solo lugar, NivelRiesgoTexto.

Las cinco reglas

Tipo	Parámetros	Qué mide
RECURRENCIA_FALLAS	minIncidencias, ventanaDias	Cantidad de fallas en una ventana
ACUMULACION_INCIDENCIAS	minIncidencias, ventanaDias, mismaCategoria	La misma falla repetida
MANTENIMIENTO_VENCIDO	diasSinMantenimiento	Días desde el último preventivo
ANTIGUEDAD_EQUIPO	aniosUmbral	Antigüedad desde la compra
GARANTIA_POR_VENCER	diasAviso	Proximidad del vencimiento

Los parámetros van en JSON en ReglaAlerta.condicion. Cada tipo define sus claves; una tabla de parámetros por tipo multiplicaría tablas sin agregar nada, porque esos valores se leen siempre junto con la regla.

Dos detalles que importan:

- MANTENIMIENTO_VENCIDO cuenta desde el último mantenimiento **preventivo**. Un correctivo es la reparación posterior a la falla: si contara, el equipo que más se rompe sería el que nunca dispara la alerta.
- MANTENIMIENTO_VENCIDO mide contra el **plan** cuando el equipo tiene uno (ContextoEquipo.Programados): «vencido» es el trabajo agendado cuya fecha ya pasó, y no una cuenta de días. Con plan, el plan manda y el umbral no vuelve a mirarse, porque un equipo con el preventivo agendado para la semana que viene no es un equipo descuidado aunque hayan pasado 250 días del anterior. El umbral queda para los equipos sin plan y, en los que tienen, para saturar el atraso: un mes de atraso pesa distinto sobre un plan mensual que sobre uno anual.
- ACUMULACION_INCIDENCIAS con mismaCategoria **no agrupa las incidencias sin clasificar**. Tres reportes sin categoría no permiten afirmar que sea la misma falla tres veces: lo que hay ahí es un dato que falta.

Un JSON roto invalida esa regla, no la evaluación del parque: la condición la edita el usuario. Además se valida al guardar, para que el error se vea al configurar y no cuando el sistema deja de avisar.

Alertas

Una alerta por regla y equipo mientras siga abierta. Sin ese control, cada evaluación duplicaría las mismas alertas y en una semana la pantalla sería inservible.

Integración de IA

IProveedorIA con dos implementaciones intercambiables por configuración.

`ProveedorIAClaude` habla con la API de Anthropic. Dos cuidados que no son evidentes:

- **Valida que el técnico recomendado esté entre los candidatos enviados.** Un modelo puede devolver un GUID bien formado que no corresponde a nadie, y asignar a un técnico inexistente deja la incidencia en un estado todavía más difícil de detectar que si no se hubiera asignado.
- **El prompt lleva los datos delimitados y marcados como datos.** La descripción de la incidencia la escribe un usuario final: es texto no confiable entrando a un prompt.

Temperatura 0, para que el mismo caso dé siempre el mismo técnico y la asignación se pueda auditar.

`ProveedorIASimulado` resuelve lo mismo con heurísticas explicables, sin red. Existe por tres razones, en orden de importancia: el sistema tiene que funcionar sin crédito de API; es lo que hace demostrable el CP-14; y es la línea base contra la cual se mide si la IA aporta algo.

Es **determinístico**: el mismo contexto da siempre el mismo resultado. El desempate por identificador no es arbitrario. Sin él, dos técnicos con el mismo puntaje se alternarían según el orden que devuelva la base.

Ninguna implementación lanza ante un fallo del servicio

Devuelven un ResultadoIA<T> con el motivo. Que el servicio externo no conteste es un caso previsto del negocio, no una excepción. Si lanzara, el registro de la incidencia se caería con él.

El origen de la decisión se registra con tres valores

IA cuando la resolvió el proveedor externo, HEURISTICA cuando la resolvió el proveedor determinístico, y RESPALDO cuando ninguno respondió. La distinción no es cosmética: registrar una decisión heurística como «IA» falsearía justamente lo que el sistema tiene que poder demostrar.

Disyuntor

Tres estados: **cerrado** las llamadas pasan; **abierto** tras cinco fallos consecutivos, y durante sesenta segundos no se intenta; **semiabierto** pasa **una sola** llamada de prueba.

El problema que resuelve no es que el servicio se cuelgue —de eso se ocupa el timeout— sino que esté consistentemente caído: sin disyuntor, cada incidencia paga el timeout completo para terminar en el mismo respaldo previsible.

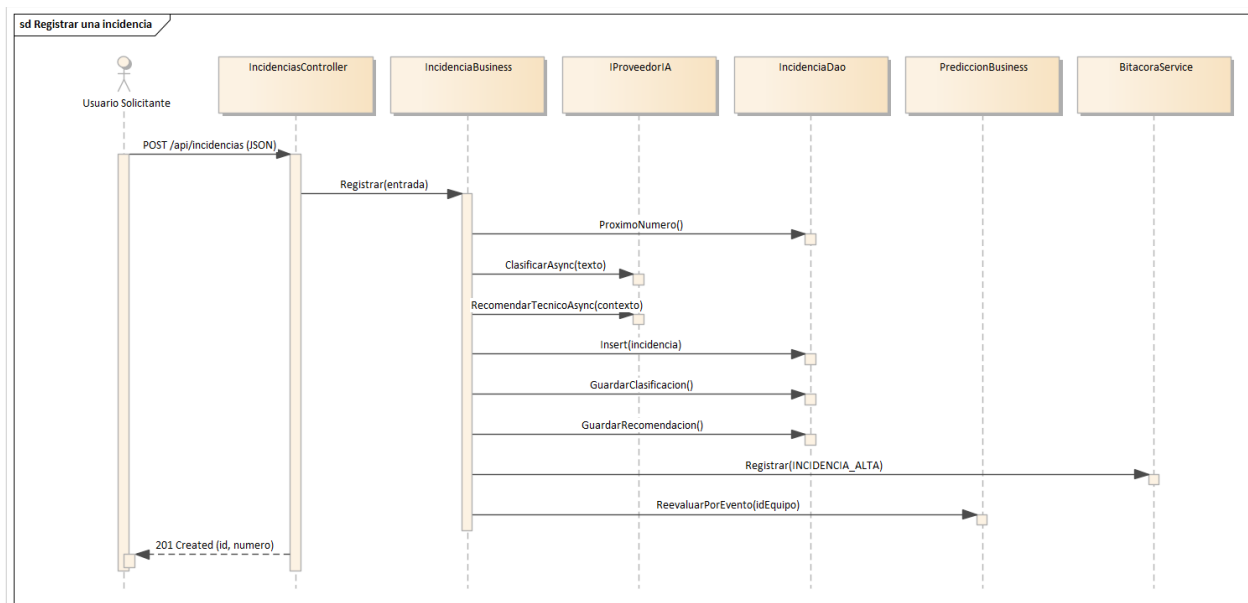
Tres cosas para tener en cuenta:

- **Es por organización, no global.** Los modos de falla del proveedor son en buena medida por cliente —clave vencida, cuenta sin crédito—, y un disyuntor compartido dejaría sin asignación automática a organizaciones cuyo proveedor responde bien.
- **Cuenta también al proveedor que lanza.** Si no, un proveedor que lanza nunca abriría el circuito y se perdería la protección.
- **Es estado en memoria de esta instancia.** Con varias instancias de la API cada una tiene el suyo. Está fuera de alcance y declarado como límite conocido.

El reloj entra por constructor para que las pruebas muevan el tiempo sin esperar sesenta segundos.

Flujo de registrar una incidencia

Es el camino más importante del sistema:



1. Se valida la entrada y se resuelve el equipo.
2. **Triage:** se clasifica el texto libre. Lo que el solicitante cargó a mano manda; la sugerencia sólo completa lo que dejó vacío.
3. **Alta,** con el correlativo y la incidencia en una transacción.
4. **Asignación:** se arma el contexto con lo que la organización autorizó enviar, se consulta al proveedor y se persiste la recomendación con el contexto exacto.
5. Si no responde, **estrategia de respaldo** y marca de revisión.

Cuatro cosas que hay que preservar:

- **Los pasos 2, 4 y 5 nunca hacen fallar el paso 3.** Que la asignación automática no funcione no puede impedir que se registre el reporte de una falla.
- **La asignación va después del alta y fuera de su transacción.** Si el proveedor tarda, no se sostiene una transacción abierta tomando bloqueos.
- **El correlativo lo calcula la base dentro de la transacción.** Leerlo y sumar uno en memoria daría el mismo número a dos altas simultáneas, y ese número es el que el cliente cita por teléfono.
- **Se guarda el JSON exacto que se envió al proveedor.** Sin eso no se puede reconstruir por qué recomendó lo que recomendó.

Frontend

Sin estado global: cada pantalla pide lo que necesita. El cliente HTTP está en `src/api/cliente.ts` y los tipos en `src/api/tipos.ts`, escritos a mano y no generados del OpenAPI, para que un desajuste aparezca al compilar.

El token va en `sessionStorage`, no en `localStorage`. Al cerrar la pestaña la sesión se termina, que es lo esperable en un equipo compartido.

La interfaz esconde lo que el usuario no puede hacer, pero la autorización la decide la API. Cada ítem de navegación declara sus patentes.

Dos cuidados en los listados:

- **Debounce de 300 ms en los buscadores.** Sin él, cada tecla dispara una consulta con LIKE sobre la tabla entera.
- **Guarda contra respuestas fuera de orden.** Cada efecto de búsqueda lleva un vigente que se apaga al limpiarse: sin eso, una respuesta lenta puede pisar el resultado de una búsqueda posterior.

Los tokens de diseño están en `src/estilos/tokens.css`, tomados literales del canvas exportado en `frontend/canvas/`.

Pruebas

```
dotnet test backend\PredictIT.Tests
cd frontend && npm test
```

362 pruebas del backend y 91 del frontend.

Área	Qué cubre
Hash de contraseñas	Los cuatro hashes literales de los datos iniciales
Composite de permisos	Anidamiento, diamante, ciclo, patente suelta
Aislamiento	Una prueba por operación
Motor predictivo	22 pruebas con fecha fija
Disyuntor y proveedores	Las tres transiciones, el proveedor que lanza
Integración de DAO	Contra SQL Server real

Las pruebas de integración van contra la base real y no contra dobles a propósito: los defectos que aparecieron en esa capa —una palabra reservada, un largo de columna, un *join* que excluía a todos los técnicos— eran **todos invisibles a la lectura del código**.

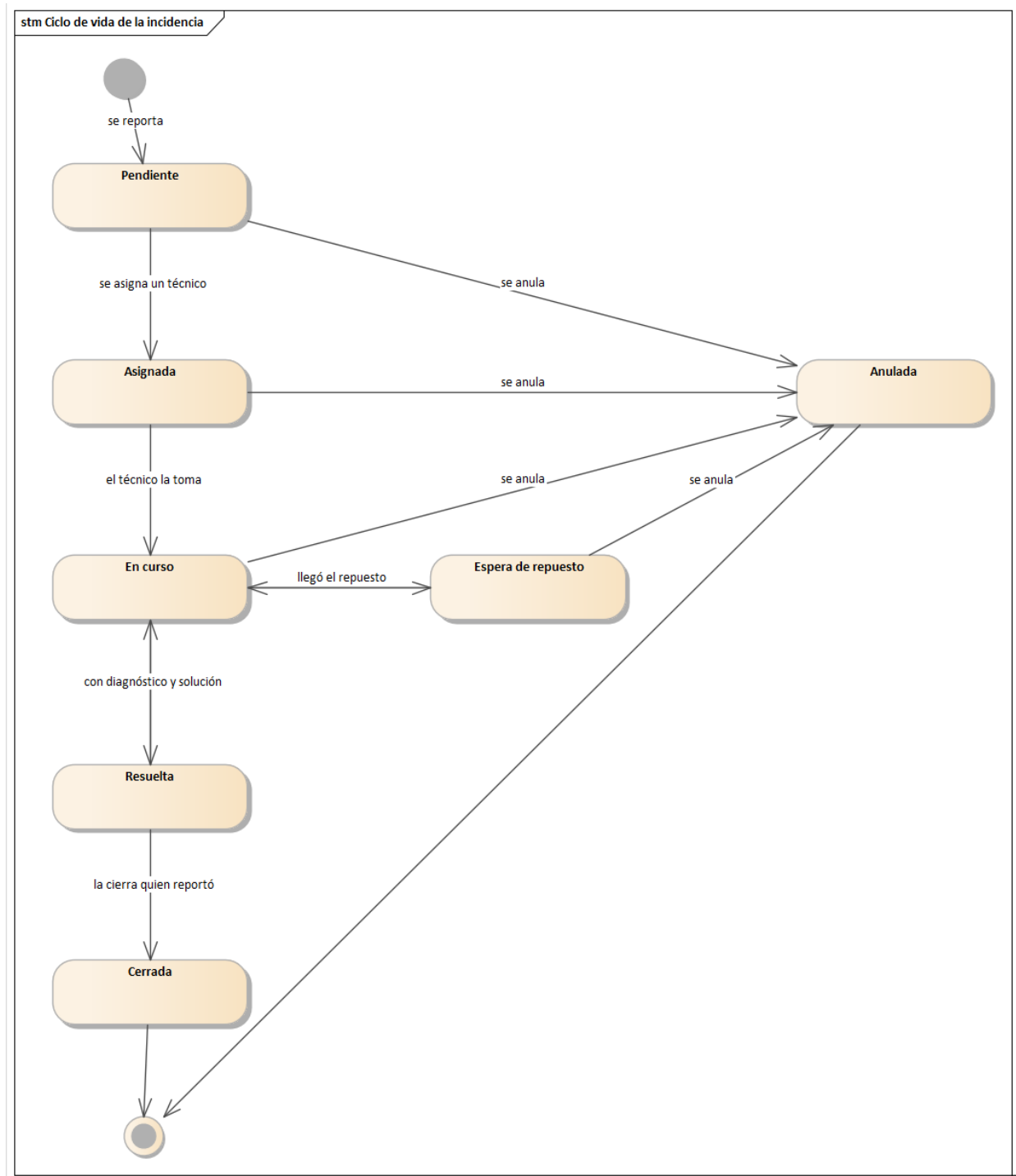
Al modificar el sistema

Una lista de las trampas que ya se pisaron:

1. **Toda consulta nueva de negocio filtra por organización.** Sin excepción.
2. **`PLAN` y `BACKUP` son palabras reservadas de T-SQL.** Las tablas son `PlanComercial` y `Respaldo`. El SQL se lee correcto y falla el parser.
3. **Verificar el largo de las columnas contra los valores que se insertan.** `VARCHAR(10)` truncando `'ADVERTENCIA'` es invisible leyendo el script.
4. **La pertenencia a una organización se expresa de dos formas.** Los usuarios internos por `Usuario.id_organizacion`; el Partner por `Usuario_Organizacion`. Mirar sólo una devuelve resultados vacíos con un SQL que se lee bien.
5. **No agregar dependencias al contexto de sesión.** Ya hubo un ciclo `IContextoSesion -> ISeguridadService -> IFactoryDao -> IContextoSesion` que compilaba, dejaba entrar al login, y colgaba el primer pedido autenticado.
6. **Los índices filtrados exigen `QUOTED\_IDENTIFIER ON`.** `sqlcmd` necesita `-I`.
7. **Correr las pruebas, no revisar el código.** Es la regla que gobierna el proyecto.

El ciclo de vida de la incidencia

La máquina de estados vive en IncidenciaBusiness y en ningún otro lado. La pantalla **consulta** qué transiciones hay —GET /incidencias/{id}/transiciones— y ofrece las que el negocio devuelve.



```

Pendiente de asignación -> Asignada (lo hace la asignación) | Anulada
Asignada                -> En curso | Anulada
En curso                -> En espera de repuesto | Resuelta | Anulada
En espera de repuesto   -> En curso | Anulada
Resuelta                -> Cerrada | En curso (reapertura)
Cerrada, Anulada        -> finales

```

Si agregás un estado o una transición, se toca un solo diccionario: Permitidas. El frontend no necesita cambios, porque no conoce la máquina.

El reparto entre validar y guiar

Esto es lo más fácil de romper del módulo, así que va explícito.

TransicionesDe informa **sólo los impedimentos que el formulario no puede conocer**: si hay técnico asignado y quién es. **No** informa que falte el diagnóstico o la solución, aunque resolver los exija.

El motivo: ese método mira lo **guardado**, y el formulario manda los textos en la misma llamada que la transición. Reportarlo dejaba el botón «Resuelta» deshabilitado para siempre —para guardar el diagnóstico hacía falta una transición, y la transición pedía el diagnóstico guardado—.

La validación de los dos textos vive en Atender, el único lugar donde se puede validar sobre los datos ya guardados, y vale para cualquier cliente. El formulario deshabilita el botón mirando lo escrito. Si "arreglás" esto moviendo la validación a las transiciones, volvés a introducir el interbloqueo.

La fecha de resolución

La escribe el paso a *Resuelta* y la **limpia** la reapertura. Es el dato del que sale el tiempo de reparación: una incidencia reabierta que conservara la fecha vieja produciría un tiempo falso, y falso hacia abajo.

Ver ADR 0012.

Reportes en PDF

IReporteService.Listado<T> recibe encabezado, filas y definición de columnas. Los tres reportes son **tres llamadas y no tres clases**: lo que cambia entre ellos son los datos y los títulos, no el documento.

Para agregar un reporte hace falta: una rama en el switch de ReporteBusiness.Generar, y la lista de ColumnaReporte<T>. Nada más.

Los reportes leen los datos que ya tiene el sistema —el parque sale del mismo listado que la pantalla de activos—. Si un reporte trajera sus propias consultas, la pantalla y el papel podrían decir cosas distintas sobre el mismo día.

Se usa **QuestPDF** con licencia Community, gratuita por debajo del millón de dólares de facturación anual. Se eligió porque no necesita binarios nativos ni un navegador headless: funciona dentro del contenedor de la API sin engordarlo.

La licencia se declara en el constructor estático de `ReporteService`. Sin esa línea, QuestPDF falla al primer documento con un mensaje que no dice que falta.

Ver ADR 0013.

Al agregar una patente

Este orden importa, y hay una razón concreta:

1. La fila en `dbo.Patente` del seed, con su `tipo_acceso` (1 lectura, 2 escritura, 3 administración).
2. La fila en `dbo.Familia_Patente` que la concede a alguna familia.
3. La constante en `Patentes (BLL/Contracts/Contratos.cs)`.
4. El `[RequierePatente(...)]` en el endpoint.
5. El `Exigir(...)` en el método de negocio.

Si te detenés en el paso 3, la patente queda declarada y sin usar. Eso pasó cuatro veces en este proyecto, y las cuatro veces significaba un caso de uso que el modelo de permisos prometía y el sistema no tenía. La forma de detectarlo es buscar los usos de cada constante fuera de su declaración:

```
grep -rn "Patentes.NombreDeLaConstante" --include=*.cs backend/
```

Si devuelve sólo la línea de la declaración, falta implementar algo.

Al agregar un código de evento a la bitácora

1. La fila en `dbo.TipoEvento` del seed.
2. La constante en `TipoEvento.Codigos (Domain/Seguridad/Bitacora.cs)`.
3. El `bitacora.Registrar(TipoEvento.Codigos.LoQueSea, ...)`.

Nunca con un literal. Una constante mal escrita no compila; un literal mal escrito sí, y `BitacoraService` descarta en silencio los códigos que no están en el catálogo. Eso pasó con

ocho de nueve códigos: la bitácora tenía los eventos de seguridad y ninguno de negocio, sin un solo error visible.

Dos pruebas lo sostienen y van a fallar si te salteás un paso:

- `BitacoraCatalogoTests.Cada_codigo_declarado_existe_en_el_catalogo` — verifica por reflexión cada constante contra la base.
- `BitacoraCatalogoTests.El_negocio_no_asienta_con_literales_sueltos` — recorre la BLL buscando `bitacora.Registrar("...")`.

Fuentes

- Anthropic. (2025). *Claude API reference*. <https://docs.anthropic.com>
- Beck, K. (2003). *Test-driven development: By example*. Addison-Wesley.
- Evans, E. (2003). *Domain-driven design: Tackling complexity in the heart of software*. Addison-Wesley.
- Fowler, M. (2002). *Patterns of enterprise application architecture*. Addison-Wesley.
- Gamma, E., Helm, R., Johnson, R. y Vlissides, J. (1995). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.
- Jones, M., Bradley, J. y Sakimura, N. (2015). *JSON Web Token (JWT) (RFC 7519)*. Internet Engineering Task Force. <https://www.rfc-editor.org/rfc/rfc7519>
- Kruchten, P. (1995). The 4+1 view model of architecture. *IEEE Software*, 12(6), 42-50.
- Martin, R. C. (2017). *Clean architecture: A craftsman's guide to software structure and design*. Prentice Hall.
- Meta Open Source. (2024). *React documentation*. <https://react.dev>
- Microsoft. (2024). *Documentación de ASP.NET Core y .NET 9*. Microsoft Learn. <https://learn.microsoft.com/aspnet/core>
- National Institute of Standards and Technology. (2017). *Digital identity guidelines: Authentication and lifecycle management* (NIST Special Publication 800-63B). <https://pages.nist.gov/800-63-3/sp800-63b.html>
- Nygard, M. T. (2018). *Release it! Design and deploy production-ready software* (2.^a ed.). Pragmatic Bookshelf.